

1. Document Control

Item	Value
Document	Software Requirements Specification
Product	Social Media Reel Saver / Media Library Application
Version	1.0
Date	24 July 2026
Primary Platforms	iOS and Android
Initial Source Platforms	Instagram and Facebook
Primary Interaction	OS Share Sheet / Share Intent
Fallback Interaction	Manual URL paste
Backend	REST API + asynchronous download worker
Database	PostgreSQL for metadata
Media Storage	Object storage such as Cloudflare R2, AWS S3, or compatible service

2. Purpose

This document defines the functional, technical, data, security, integration, and user-experience requirements for a mobile application that receives Instagram or Facebook reel/video links through the operating-system share sheet or manual URL entry, processes the link through a backend downloader pipeline, stores the downloaded media in managed object storage, records metadata in PostgreSQL, and presents the saved media inside the application library.

3. Product Scope

The MVP will allow a user to share a supported public or otherwise legitimately accessible Instagram/Facebook media link to the application. The application will automatically detect the platform, submit the link, show download status, and display the media in a persistent library once processing succeeds. Manual URL paste is retained as a fallback. The system is designed so additional providers can be added later without changing the mobile contract.

4. Goals and Non-Goals

4.1 Goals

- Make saving a reel/video a low-friction action: Share -> select the app -> automatic processing.
- Keep long-running download work outside the mobile UI and outside synchronous API requests.
- Persist media metadata, source platform, source URL, timestamps, status, and storage location.
- Provide a clean media library with playback, filtering, retry, and deletion capabilities.
- Keep the download provider replaceable through a provider/extractor abstraction.
- Support future platforms such as TikTok, X, YouTube, or Pinterest without redesigning the core pipeline.

4.2 Non-Goals for MVP

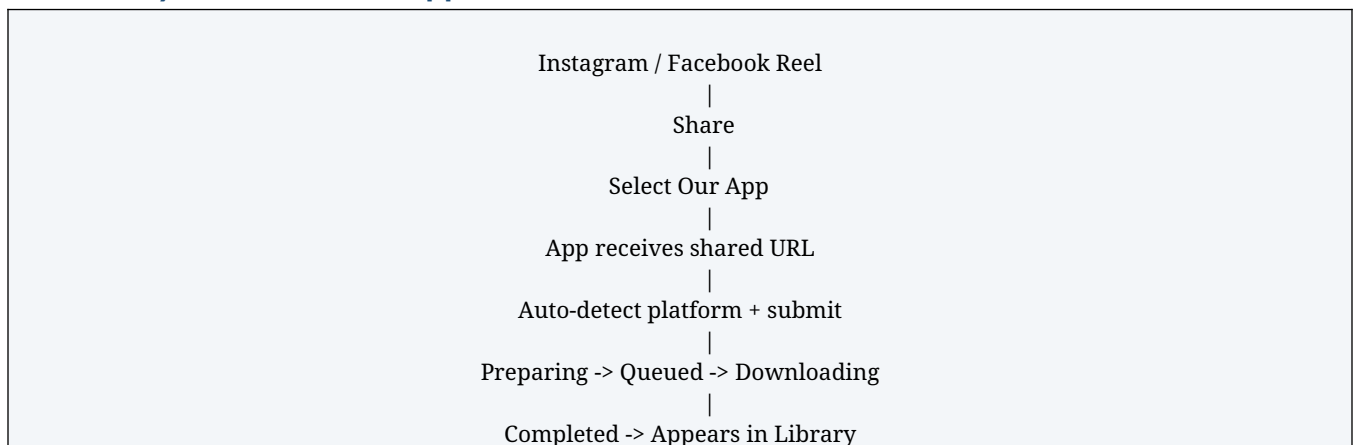
- Social-network account publishing, commenting, liking, or analytics.
- Downloading content that the user is not authorized or permitted to access.
- Editing or transcoding videos beyond optional thumbnail generation and basic validation.
- Storing large video binaries directly inside PostgreSQL.

5. Users and Actors

Actor	Responsibility
End User	Shares or pastes a media URL, views status, plays, retries, or deletes saved media.
Mobile Application	Receives shared URL, calls backend APIs, shows local/realtime state, plays media.
Backend API	Authenticates user, validates request, creates media record, queues jobs, exposes status/library APIs.
Downloader Service	Consumes jobs, resolves media, downloads content, uploads files, updates progress and result.
PostgreSQL	Stores user and media metadata, status, timestamps, storage keys, errors, and audit fields.
Object Storage	Stores video files and thumbnails.
Queue / Broker	Decouples API requests from downloading and supports retries/concurrency.

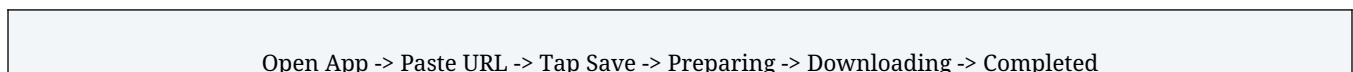
6. User Experience and Primary Workflows

6.1 Primary Flow - Share to App



- User should not be required to manually copy and paste the URL in the primary flow.
- On Android, the app shall register an appropriate Share Intent for shared text/URLs.
- On iOS, the product shall provide a Share Extension / share-sheet entry capable of passing the URL to the main app or backend flow.
- When a valid supported URL is received, the app shall start the save workflow with minimal additional interaction.

6.2 Fallback Flow - Manual URL Paste



6.3 Download Status UI

Status	User-facing behavior
PREPARING	App validates shared/pasted URL and creates a media item.
QUEUED	Media item is accepted and waiting for a worker.
DOWNLOADING	Download is active; show progress when reliable progress

	data is available.
PROCESSING	Optional validation/thumbnail/upload finalization stage.
COMPLETED	Show thumbnail/player, platform, save/download time, and actions.
FAILED	Show a clear error state with Retry and Delete actions when appropriate.

7. Mobile UI Requirements

Screen	Required Elements
Home	Share-entry handling, manual URL field, Save button, recent downloads, pending/failed cards.
Downloads / Library	Grid/list of saved media, platform filter, search/sort, status indicators.
Media Detail	Thumbnail/player, source platform, saved date/time, original URL, file details, delete action.
Download Item / Progress	Preparing/queued/downloading/processing/completed/failed state and retry where supported.
Settings	Account/session controls, storage/cache information, legal/privacy links, optional download preferences.

8. Functional Requirements

FR-001 - Receive Shared URL

The mobile application shall accept URLs shared from supported applications through the operating-system share mechanism.

Acceptance: Sharing a supported Instagram/Facebook URL opens or invokes the application and supplies the URL without requiring manual paste.

FR-002 - Manual URL Entry

The mobile application shall provide a URL input as a fallback.

Acceptance: A valid supported URL entered manually can be submitted with the Save action.

FR-003 - Platform Detection

The system shall determine the source platform from the submitted URL.

Acceptance: Instagram and Facebook URLs are classified correctly; unsupported domains are rejected with a useful message.

FR-004 - URL Validation

The backend shall validate syntax, scheme, supported host/domain, and request constraints before queueing.

Acceptance: Malformed or unsupported URLs never create a download job.

FR-005 - Create Media Record

The backend shall create a persistent media record before asynchronous processing starts.

Acceptance: Every accepted request has a unique media ID and initial status.

FR-006 - Queue Download Job

The backend shall enqueue download work instead of downloading within the request thread.

Acceptance: Create-media API returns promptly with a media ID and queued/preparing status.

FR-007 - Resolve Media Source

The downloader service shall use a platform-specific provider/extractor to resolve a usable media source for permitted content.

Acceptance: Worker either obtains a valid media source or records a structured failure.

FR-008 - Download Media

The worker shall download media to temporary/streaming storage with configured size and timeout limits.

Acceptance: Successful jobs produce a validated media file without exhausting API server resources.

FR-009 - Upload Media

The worker shall upload completed media to object storage and record the storage key.

Acceptance: Completed records reference an existing object-storage key.

FR-010 - Generate Thumbnail

The system should generate or retain a thumbnail when feasible.

Acceptance: Completed items display a thumbnail; absence of thumbnail does not block successful save.

FR-011 - Persist Metadata

The system shall store platform, original URL, status, created time, download times, storage key, file size, and failure information.

Acceptance: Metadata remains available after app restart/login.

FR-012 - Progress / Status

The mobile app shall receive or retrieve the latest processing status.

Acceptance: A user can see whether a media item is queued, downloading, completed, or failed.

FR-013 - Library Listing

The backend shall provide paginated media-library results for the authenticated user.

Acceptance: Library loads saved items ordered by most recent by default.

FR-014 - Media Playback

The user shall be able to play successfully saved media from the application.

Acceptance: Tapping a completed item opens a playable media view using an authorized delivery URL.

FR-015 - Retry Failed Download

Eligible failed jobs shall be retryable.

Acceptance: Retry creates/requeues work without creating duplicate completed media accidentally.

FR-016 - Delete Media

The user shall be able to delete a saved item.

Acceptance: Deletion removes or schedules removal of the object and removes/hides the library record according to retention policy.

FR-017 - Duplicate Handling

The system should detect repeated save requests for the same user/source URL.

Acceptance: Behavior is deterministic: reuse existing completed item or create a new version according to configured policy.

FR-018 - Provider Abstraction

Downloader logic shall be accessed through a provider/extractor interface rather than hard-coded into API controllers.

Acceptance: A provider can be replaced without changing mobile API contracts.

9. Proposed Backend API

Method	Endpoint	Purpose
POST	/api/v1/media	Create/save a media item from shared or pasted URL.
GET	/api/v1/media	List authenticated user media with pagination/filtering.
GET	/api/v1/media/{id}	Get item metadata and current status.
POST	/api/v1/media/{id}/retry	Retry an eligible failed item.
DELETE	/api/v1/media/{id}	Delete item and associated managed media.
GET	/api/v1/media/{id}/playback	Return an authorized/signed playback URL if applicable.

9.1 Create Media - Example

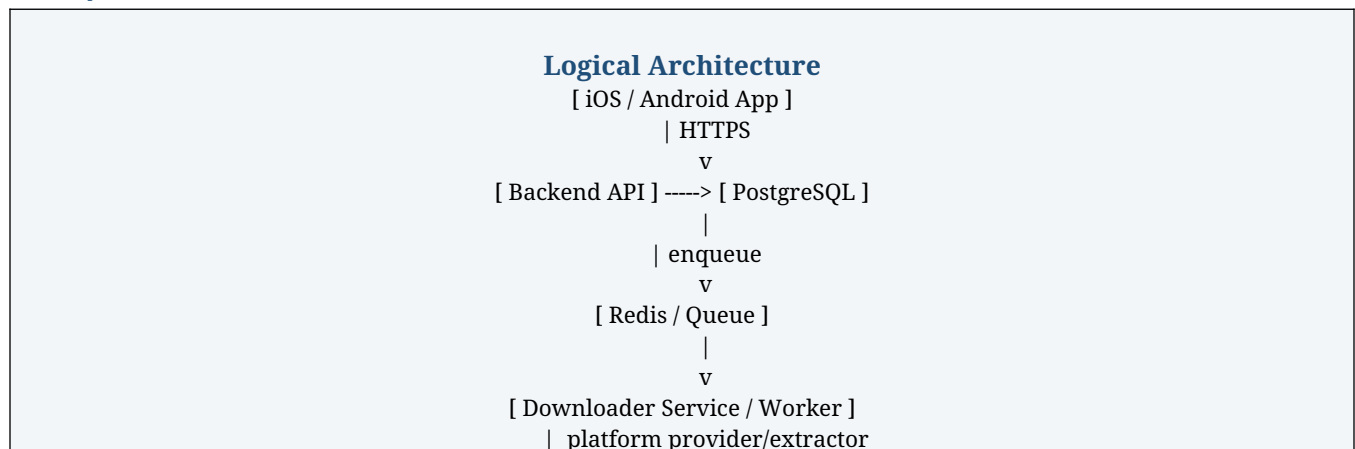
POST /api/v1/media

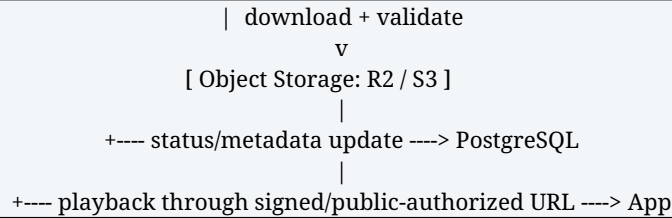
```
{
  "url": "https://www.instagram.com/reel/ABC123/",
  "source": "share_sheet"
}
```

202 Accepted

```
{
  "id": "<uuid>",
  "platform": "instagram",
  "status": "queued",
  "createdAt": "2026-07-24T16:35:00+04:00"
}
```

10. System Architecture



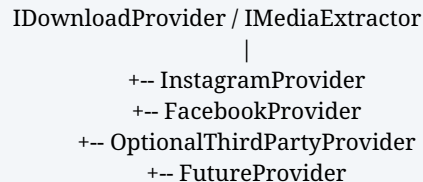


10.1 Component Responsibilities

Component	Responsibilities
Mobile App	Share target/extension, URL capture, API calls, progress display, library, playback, retry/delete.
Backend API	Authentication, validation, media-record lifecycle, queue publishing, authorization, library APIs.
Queue	Reliable asynchronous job delivery, retry scheduling, worker concurrency control.
Downloader Service	Platform adapter selection, media resolution, download, validation, thumbnail, upload, status updates.
Provider/Extractor Layer	Instagram/Facebook-specific resolution logic behind a common contract.
PostgreSQL	Transactional metadata and state.
Object Storage	Large media binaries and thumbnails.

11. Downloader Service Design

The downloader service is a backend worker and is not exposed directly to the mobile UI. It consumes queued jobs and implements platform-specific behavior through replaceable providers.



Worker Step	Description
1. Consume	Read a media job from the queue.
2. Load record	Load media item and verify it is eligible to process.
3. Select provider	Choose provider using normalized platform classification.
4. Resolve	Resolve media metadata/source subject to access and policy rules.
5. Download	Stream media using configured timeout, size, and network limits.
6. Validate	Verify content type/format and basic file integrity.
7. Thumbnail	Generate or save thumbnail when available.
8. Upload	Upload final object(s) to object storage.
9. Complete	Update PostgreSQL status, timestamps, size, keys, and error fields.

12. Data Model

12.1 media_items

Column	Type	Notes
id	UUID	Primary key.
user_id	UUID	Owner / authorization boundary.
original_url	TEXT	User-shared or pasted source URL.
normalized_url	TEXT	Optional canonicalized URL for duplicate detection.
platform	VARCHAR/ENUM	instagram, facebook, future providers.
status	VARCHAR/ENUM	preparing, queued, downloading, processing, completed, failed.
title	TEXT	Optional extracted title/caption summary.
thumbnail_storage_key	TEXT	Object storage key for thumbnail.
media_storage_key	TEXT	Object storage key for primary media.
mime_type	VARCHAR	Example: video/mp4.
file_size_bytes	BIGINT	Final media size.
duration_ms	BIGINT	Optional duration.
progress_percent	SMALLINT	Optional 0-100; not authoritative for completion.
created_at	TIMESTAMPTZ	Request accepted time.
download_started_at	TIMESTAMPTZ	Worker start.
downloaded_at	TIMESTAMPTZ	Download/upload completion time.
updated_at	TIMESTAMPTZ	Last state change.
error_code	VARCHAR	Structured failure identifier.
error_message	TEXT	Sanitized user/support diagnostic.
retry_count	INTEGER	Attempt count.

12.2 Recommended Indexes

- Index on (user_id, created_at DESC) for library queries.
- Index on (user_id, status) for pending/failed filters.
- Optional unique/partial policy on (user_id, normalized_url) when duplicate prevention is enabled.
- Index on status / next_retry_at if workers query the database in addition to the queue.

13. Media State Machine

```

PREPARING -> QUEUED -> DOWNLOADING -> PROCESSING -> COMPLETED
                                     \
                                     > FAILED
FAILED --retry--> QUEUED

```

Only valid state transitions should be permitted. COMPLETED is terminal unless the user deletes the item; FAILED may be retried based on error type and retry policy.

14. Error Handling and Retry Policy

Error Category	Example Behavior
INVALID_URL	Reject before queueing; show unsupported/invalid link.
UNSUPPORTED_PLATFORM	Reject before queueing.
ACCESS_NOT_PERMITTED	Fail without bypassing access controls; explain that content cannot be retrieved.

MEDIA_NOT_FOUND	Fail; allow retry only if transient behavior is plausible.
PROVIDER_TEMPORARY_FAILURE	Retry with exponential backoff and capped attempts.
DOWNLOAD_TIMEOUT	Retry according to configured policy.
FILE_TOO_LARGE	Fail without retry; show configured limit.
STORAGE_FAILURE	Retry upload without re-resolving media when safe.
UNKNOWN	Record sanitized diagnostic, correlation ID, and cap retry attempts.

15. Non-Functional Requirements

ID	Requirement
NFR-001	Create-media API should normally acknowledge accepted requests quickly and must not wait for full video download.
NFR-002	Worker concurrency shall be configurable independently from API replicas.
NFR-003	All external network requests shall have connect/read timeouts and bounded retries.
NFR-004	Object storage shall be used for large media; PostgreSQL shall store metadata rather than video BLOBs for the normal architecture.
NFR-005	APIs shall enforce authenticated ownership for read, playback, retry, and delete operations.
NFR-006	Logs shall include correlation/media/job identifiers but must not expose credentials, session secrets, or unsafe signed URLs.
NFR-007	System shall support horizontal worker scaling.
NFR-008	The media library shall use pagination or cursor-based loading.
NFR-009	Download size, duration, bandwidth, and concurrency limits shall be configurable.
NFR-010	Service shall expose health checks and basic operational metrics.

16. Security, Privacy, and Platform Compliance

- The backend shall validate URLs and defend against SSRF by restricting outbound resolution/download targets to approved platform/provider flows and safe resolved hosts.
- The system shall not attempt to bypass authentication, paywalls, DRM, private-account controls, or other access restrictions.
- Users should save only content they are authorized to access and store; product terms and UX should communicate this responsibility.
- Credentials, provider tokens, cookies, and API keys must remain server-side and must not be embedded in the mobile application.
- Playback URLs should be access-controlled or time-limited when private object storage is used.
- Delete operations should remove managed media according to the product retention policy.
- Rate limiting and abuse controls should be applied per user/IP/device where appropriate.
- Platform-specific implementation must be reviewed against applicable Instagram/Facebook/Meta terms and legal requirements before production launch.

17. Status Synchronization

Two implementation levels are acceptable:

Option	Use
MVP Polling	App calls GET /media/{id} every few seconds while an item is active. Simple and reliable for first release.
Realtime	SignalR/WebSocket/SSE pushes status changes. Recommended once scale and UX justify it.

18. Recommended Technology Baseline

Layer	Recommendation
Mobile	Flutter (single iOS/Android codebase) or native equivalent.
API	ASP.NET Core Web API.
Worker	Separate .NET Worker Service or dedicated service/runtime suitable for provider implementation.
Database	PostgreSQL.
Queue	Redis-based queue for MVP; RabbitMQ/SQS/Service Bus are valid alternatives.
Object Storage	Cloudflare R2, AWS S3, MinIO, or S3-compatible storage.
Realtime	SignalR if using ASP.NET Core.
Deployment	Docker containers behind a reverse proxy/load balancer.
Observability	Structured logs, error tracking, job metrics, health endpoints.

19. End-to-End Sequence

1. User taps Share in Instagram/Facebook.
2. User selects the application from the share sheet.
3. App receives the shared URL.
4. App calls POST /api/v1/media.
5. API validates URL, creates DB record, enqueues job.
6. API returns media ID + QUEUED status.
7. Worker consumes job and sets DOWNLOADING.
8. Provider resolves permitted media source.
9. Worker downloads and validates media.
10. Worker uploads video/thumbnail to object storage.
11. Worker updates DB to COMPLETED with storage metadata.
12. App receives/polls status and updates UI.
13. Item appears in Library and can be played.

20. MVP Acceptance Criteria

- AC-01: A user can share a supported Instagram URL to the app and the URL is received automatically.
- AC-02: A user can share a supported Facebook URL to the app and the URL is received automatically.
- AC-03: A user can paste a supported URL manually when share integration is unavailable.
- AC-04: Accepted requests create a persistent media item and asynchronous job.
- AC-05: The API does not wait for full media download before responding.
- AC-06: The app displays queued/downloading/completed/failed state.

- AC-07: A successful media file is stored in object storage, not in PostgreSQL.
- AC-08: PostgreSQL contains platform, source URL, timestamps, status, storage key, and error/retry metadata.
- AC-09: Completed media appears in the authenticated user library and can be played.
- AC-10: Failed transient jobs can be retried and permanent failures are clearly reported.
- AC-11: A user cannot access another user's media by guessing an ID.
- AC-12: Deleting an item removes or schedules deletion of the associated managed object.

21. Future Enhancements

- Additional social platforms through new provider adapters.
- Collections/folders/tags and favorites.
- Background notifications when downloads complete.
- Offline device cache in addition to server-side library storage.
- User-configurable quality/format where legally and technically supported.
- Duplicate-content hashing and storage deduplication.
- Admin dashboard for queue health, failures, provider performance, and storage usage.
- Automatic provider fallback when multiple compliant resolution providers are configured.

22. Open Product / Engineering Decisions

Decision	Recommended Starting Point
Authentication	JWT + refresh tokens or existing identity provider.
Auto-start after share	Yes for a valid supported URL; show immediate cancel where practical.
Duplicate URL behavior	Return existing completed item for same user, configurable later.
Maximum media size	Set a product limit before launch based on storage/bandwidth budget.
Progress accuracy	Use indeterminate state when reliable byte progress is unavailable.
Polling vs realtime	Polling for MVP; SignalR after core flow is stable.
Storage privacy	Private bucket + signed playback URLs recommended.
Downloader provider	Provider abstraction required; implementation selected during spike/prototype.

Appendix A - Suggested Project Structure

```
src/
  MobileApp/
  Backend.Api/
    Controllers/
    Application/
    Domain/
    Infrastructure/
  Downloader.Worker/
  Jobs/
  Providers/
    IMediaProvider
    InstagramProvider
    FacebookProvider
```

Downloading/
Storage/
Shared.Contracts/
Database/
Deployment/

Appendix B - Definition of Done for a Download

- The item belongs to the authenticated user.
- The source URL and platform are persisted.
- The final media object exists in managed object storage.
- The database contains the correct media storage key and file metadata.
- Status is COMPLETED and downloaded_at is populated.
- The mobile app can request an authorized playback URL and play the media.
- No sensitive provider credentials are exposed to the client.

END OF DOCUMENT